

Internal Task & Project Management Platform

Project Type: Production-Level Full-Stack SaaS Application

Reference Products: Jira / ClickUp / Linear

Purpose: Internal company project, task, team, and workflow management

Important Project Standard

This project must be developed as a production-level application, not as a college project or basic internship CRUD project. Think like a professional software engineer:

Requirement → Architecture → Database → API → Frontend → Security → Testing → CI/CD → Cloud Deployment → Monitoring → Documentation

The final application should be maintainable, scalable, secure, testable, and deployable.

Table of Contents

1. [Project Objective](#)

2. [User Roles](#)
3. [Authentication](#)
4. [Organization / Workspace](#)
5. [Team Management](#)
6. [Project Management](#)
7. [Task Management](#)
8. [Subtasks](#)
9. [Task Views](#)
10. [Search and Filtering](#)
11. [Comments](#)
12. [Attachments](#)
13. [Notifications](#)
14. [Activity / Audit History](#)
15. [Dashboard](#)
16. [Reports & Analytics](#)
17. [Backend Requirements](#)
18. [Database Requirements](#)
19. [API Standards](#)
20. [Error Handling](#)
21. [Error Logs & Application Logging](#)
22. [Security Requirements](#)
23. [Frontend Requirements](#)
24. [Admin Panel](#)
25. [Testing](#)
26. [Git & Development Workflow](#)

- 27. [Environment Management](#)
 - 28. [Cloud Deployment](#)
 - 29. [CI/CD](#)
 - 30. [Database Backup & Recovery](#)
 - 31. [Performance](#)
 - 32. [Documentation](#)
 - 33. [Production Readiness Checklist](#)
 - 34. [Final Acceptance Criteria](#)
-

1. Project Objective

Build a production-level internal project and task management platform that allows the company to manage:

- Teams
- Employees
- Projects
- Tasks & Subtasks
- Project workflows
- Deadlines
- Team activities
- Comments & Attachments
- Notifications
- Project progress

- Reports and analytics

The application should be designed as a real production-ready SaaS-style platform — **not** a basic CRUD or internship demo. Every feature must be implemented with proper architecture, validation, authorization, security, error handling, logging, testing, documentation, and cloud deployment.

2. User Roles

The system must support role-based access control (RBAC).

Role	Capabilities
Super Admin	Manage entire platform, users, teams, roles & permissions, projects, company-wide analytics, audit logs, system configuration
Admin / Project Manager	Create projects, manage project members, create/assign tasks, manage workflows, view project analytics, manage deadlines, monitor progress

Role	Capabilities
Team Lead	Manage team tasks, assign tasks, review team progress, update task status, view team reports
Employee	View assigned projects, create/update permitted tasks, comment, upload attachments, update task status, track personal workload

3. Authentication

Implement a production-level authentication system.

Required features:

- Registration / invitation
- Login / Logout
- Forgot password / Reset password
- Email verification
- Change password
- Session/token management
- Role-based authorization
- Account activation / deactivation
- Protected routes

- Secure password hashing
- Authentication failure handling

Passwords must never be stored in plain text.

4. Organization / Workspace

The platform should support company workspaces.

```
Workspace
├── Users
├── Teams
├── Projects
├── Tasks
├── Notifications
└── Activity
```

Required:

- Workspace creation & settings
 - Member management & invitations
 - Role assignment / removal
 - Workspace-level permissions
-

5. Team Management

- Create / Update / Delete / Archive team

- Add / Remove members
 - Assign team lead
 - Team dashboard, workload, and task statistics
-

6. Project Management

Project fields:

Name, Description, Owner, Team, Members, Start date, End date, Status, Priority, Tags, Progress, Created date, Updated date

Project statuses: Planning → Active → On Hold → Completed → Archived

7. Task Management

Tasks are the core feature. Each task should support:

Title, Description, Project, Assignee, Reporter, Priority, Status, Due date, Start date, Labels, Estimated effort, Actual effort, Subtasks, Dependencies, Comments, Attachments, Activity history

Priority levels: Low · Medium · High · Critical

8. Subtasks

A task can contain multiple subtasks, e.g.:

```
Build Authentication
|
├─ Create database entities
├─ Implement registration
├─ Implement login
├─ Implement JWT
├─ Implement password reset
└─ Write authentication tests
```

Track: Completion, Assignee, Status, Due date

9. Task Views

View	Description
List View	Sortable and filterable task list
Kanban View	Columns: TODO → IN PROGRESS → REVIEW → DONE
Calendar View	Tasks displayed by deadline
My Tasks	Tasks assigned to the current employee

10. Search and Filtering

Implement global and project-level search, optimized for production use (avoid loading unnecessary records).

Filters: Assignee, Project, Team, Status, Priority, Due date, Labels, Created date

11. Comments

- Add / Edit / Delete comment (where permitted)
 - Timestamps and user info
 - Mention users
 - Comment activity
-

12. Attachments

- File type & size validation
 - Secure storage via cloud/object storage (not stored directly on the app server in production)
 - Access control & download authorization
 - Upload error handling
-

13. Notifications

Triggered when:

- Task assigned or status changes
- User mentioned
- Task deadline approaching
- User added to a project
- Comment added
- Project status changes

Support: In-app notifications, read/unread status, notification history. Email notifications are an advanced/optional feature.

14. Activity / Audit History

Important actions must be recorded, e.g.:

```
Srikanth assigned Task #123 to Rahul  
Rahul changed Task #123 from TODO → IN  
PROGRESS  
Anil added a comment  
Project status changed to Completed
```

Record: User, Action, Entity, Entity ID, Timestamp, Relevant metadata

Audit records should not be casually editable or deletable.

15. Dashboard

Admin Dashboard: Total employees, total teams, active/completed projects, open/overdue tasks, task distribution, project progress, team performance

Employee Dashboard: My projects, my tasks, overdue tasks, upcoming deadlines, recent activity, task statistics

16. Reports & Analytics

- Project completion & progress
- Task completion & status distribution
- Overdue tasks
- Team / employee workload
- Productivity trends

Charts should be based on actual database data.

17. Backend Requirements

Clean modular architecture with modules for: `auth` ,
`users` , `workspaces` , `teams` , `projects` , `tasks` ,
`subtasks` , `comments` , `attachments` ,
`notifications` , `activity` , `reports` , `admin`

Requirements:

- REST APIs (or clearly documented API architecture)
 - DTO/request validation
 - Authentication middleware & authorization guards
 - Centralized error handling
 - Database transactions where required
 - Pagination, filtering, sorting
 - Logging & API documentation
 - Unit and integration tests
-

18. Database Requirements

Design a normalized relational database.

Key entities: User, Role, Permission, Workspace, WorkspaceMember, Team, TeamMember, Project, ProjectMember, Task, SubTask, TaskDependency, Comment, Attachment, Notification, ActivityLog

Properly define primary/foreign keys, relationships, constraints, indexes, timestamps, and soft deletion

where appropriate. Do not create the database as one large table.

19. API Standards

Every API should have proper HTTP methods, consistent response format, proper status codes, validation, auth, error responses, and pagination.

```
GET    /api/projects
POST   /api/projects
GET    /api/projects/:id
PATCH /api/projects/:id
DELETE /api/projects/:id
```

API documentation must be maintained throughout development.

20. Error Handling

Do not expose internal errors to users.

Example user-facing response: *"Something went wrong while updating the task. Please try again."*

Handle: Validation, authentication, authorization, database, file upload, external service, network, and

unexpected errors. Internal logs should contain useful debugging information.

21. Error Logs & Application Logging

Implement centralized, structured, searchable logging.

Capture: Timestamp, log level, request ID, user ID (where appropriate), API endpoint, HTTP method, status code, error message, stack trace, relevant metadata

Log levels: DEBUG · INFO · WARN · ERROR

Never log: passwords, authentication secrets, tokens, private keys, or sensitive personal information.

22. Security Requirements

Password hashing · Secure authentication & authorization · Input validation · SQL injection protection · XSS protection · CSRF protection (where applicable) · Rate limiting · Secure headers · CORS configuration · File upload security · Environment variables & secret management · Access control · Audit logging

Never commit secrets to Git.

23. Frontend Requirements

Scalable, feature-based architecture:

```
features/  
├─ auth/  
├─ users/  
├─ projects/  
├─ tasks/  
├─ teams/  
├─ notifications/  
├─ dashboard/  
└─ reports/
```

Requirements: Responsive UI, loading/empty/error states, form validation, reusable components, protected routes, permission-based UI, API integration, proper state management, accessible UI

24. Admin Panel

User management · Role management · Permission management · Team management · Project overview · Activity logs · System statistics · Account management

25. Testing

Type	Covers
Unit	Services, business logic, utilities, validation
Integration	Authentication, project creation, task creation, task assignment, permissions, notifications
API	Every major API endpoint
End-to-End	Critical user journeys, e.g. Login → Create Project → Add Team Member → Create Task → Assign Task → Update Status → Complete Task

26. Git & Development Workflow

Branch strategy: `main` · `develop` · `feature/*` · `bugfix/*` · `hotfix/*`

Requirements: Meaningful commits, pull requests, code reviews, no direct uncontrolled production changes, proper README, environment documentation

27. Environment Management

Maintain separate **Development**, **Staging**, and **Production** environments where practical.

Use environment variables for: database credentials, JWT secrets, API keys, storage credentials, email credentials, third-party services. Never hardcode secrets.

28. Cloud Deployment

```
Frontend → Cloud Hosting → Backend API →  
Database → Object/File Storage
```

Must understand: build process, environment variables, database deployment & migrations, production configuration, domain configuration, HTTPS, logs, health checks, deployment troubleshooting

29. CI/CD

```
Push Code → Install Dependencies → Lint →  
Run Tests → Build → Deploy
```

Production deployment should not depend on manually copying code to a server.

30. Database Backup & Recovery

Document: backup frequency, retention, recovery procedure, migration strategy

31. Performance

Database indexes · Pagination · Query optimization · Lazy loading · Caching where appropriate · API response optimization · Image/file optimization · Efficient frontend rendering

Do not fetch thousands of records unnecessarily.

32. Documentation

Final repository must contain:

- README.md
- API Documentation
- Architecture Documentation
- Database Documentation
- Environment Setup

- Deployment Guide
- Testing Guide
- Troubleshooting Guide

README must explain how another developer can run the project from scratch.

33. Production Readiness Checklist

- ☐ Authentication completed
- ☐ Authorization completed
- ☐ RBAC completed
- ☐ Database properly designed
- ☐ APIs documented
- ☐ Frontend responsive
- ☐ Admin panel completed
- ☐ Error handling implemented
- ☐ Application logging implemented
- ☐ Audit logging implemented
- ☐ Security review completed
- ☐ Unit tests completed
- ☐ Integration tests completed
- ☐ E2E tests for critical flows
- ☐ CI/CD configured
- ☐ Cloud deployment completed

- ☐ HTTPS configured
 - ☐ Environment variables configured
 - ☐ Database backup strategy documented
 - ☐ README completed
 - ☐ Deployment documentation completed
-

34. Final Acceptance Criteria

The project is considered complete only when it operates as a **production-level application**. A working UI alone is not sufficient. The following must be demonstrated:

1. Complete authentication
2. Role-based access
3. Project management
4. Task management
5. Kanban workflow
6. Team management
7. Comments and attachments
8. Notifications
9. Activity/audit logs
10. Dashboard and analytics
11. Secure APIs
12. Proper database architecture

- 13. Error handling
- 14. Application logging
- 15. Automated testing
- 16. Cloud deployment
- 17. CI/CD
- 18. Documentation